Documentación Técnica y Guía de Sustentación: Algoritmo Google PageRank

Asignatura: Procesos Estocásticos (Código 603504) - Unidad: Cadenas de Markov
Programa: Ingeniería de Sistemas (5.º Semestre), Universidad de los Llanos
Semestre Académico: 2026-II
Objetivo del Documento: Servir como guía integral de la arquitectura del software, estrategia de sustentación en tablero/software y solucionario argumentado del cuestionario teórico de evaluación.

1. Arquitectura y Documentación Modular del Código Python

El software está estructurado de manera modular en cinco secciones claramente definidas para garantizar la separación de responsabilidades, la mantenibilidad y el rigor matemático exigido por la guía de laboratorio.

Módulo	Componentes Principales	Función y Fundamento Matemático
Módulo 1	construir_matriz_google metodo_potencia monte_carlo_surfer	Modelado markoviano núcleo. Construcción de matrices P y M = d P + (1-d)(1/m)E, solución determinista por potencia matricial y simulación estocástica del navegante aleatorio.
Módulo 2	ExploradorHTML extraer_enlaces_html construir_red_desde_html	Extracción dinámica de hipervínculos HTML procesando etiquetas <a href="..."> sin recibir

Módulo	Componentes Principales	Función y Fundamento Matemático
		matrices precalculadas.
Módulo 3	detectar_dead_ends detectar_enlaces_duplicados detectar_spider_traps calcular_segundo_autovalor	Análisis estructural de anomalías (callejones sin salida, duplicados, trampas de araña mediante Tarjan SCC) e inspección espectral (|lambda_2|).
Módulo 4	dibujar_convergencia dibujar_comparacion dibujar_sensibilidad dibujar_ranking mostrar_dashboard_integrado	Visualización gráfica interactiva consolidada en un Dashboard 2x2 para las 4 gráficas obligatorias del laboratorio.
Módulo 5	crear_topologia_1, 2, 3 analizar_sensibilidad_d ejecutar_topologia ejecutar_laboratorio	Definición de escenarios de prueba (3, 5 y 10 nodos) y pipeline de ejecución automatizado.

1.1 Descripción Detallada de Funciones Núcleo

1. construir_matriz_google(diccionario_red, d=0.85):

Recibe un diccionario donde las llaves son URLs/nodos y los valores son listas de destinos.

- Calcula la dimensión m = |V| y asigna un índice entero a cada nodo.
- **Reparación de Dead Ends:** Si la lista de enlaces salientes de un nodo i está vacía (c_i = 0), asigna la fila como P[i, :] = 1/m, redistribuyendo de forma uniforme la probabilidad.
- **Enlaces Duplicados:** Si hay enlaces repetidos hacia una misma URL objetivo, el conteo len(enlaces) los contabiliza, incrementando proporcionalmente el peso w_ij y la probabilidad p_ij = w_ij / c_i.
- **Matriz de Google:** Aplica la transformación M = d * P + (1-d) * (1/m) * E, donde E es la

matriz de unos de m x m.

2. metodo_potencia(M, tol=1e-8, max_iter=500):
Implementa la solución determinista iterando pi_{n+1} = pi_n * M, iniciando desde una distribución uniforme pi_0 = [1/m, ..., 1/m]. Verifica la condición de paro mediante la norma L1: ||pi_{n+1} - pi_n||_1 < tol. Devuelve el vector estacionario final pi y la matriz de historial de convergencia.

3. monte_carlo_surfer(M, pasos=150000):
Simula la trayectoria de un navegante aleatorio a través de una Cadena de Markov. En cada paso, incrementa el contador de visitas del estado actual y transiciona al siguiente estado eligiendo según la distribución de probabilidades de la fila M[estado, :] con np.random.choice. Retorna la frecuencia relativa de visitas, la cual converge a pi por la Ley de los Grandes Números.

4. Detección de Spider Traps con Algoritmo de Tarjan (detectar_spider_traps):
En lugar de heurísticas simples que fallan en grafos complejos, esta función encuentra las Componentes Fuertemente Conexas (SCC) del grafo dirigidas por el algoritmo de Tarjan. Una SCC es confirmada como *Spider Trap* si es un subconjunto estricto de la red (|SCC| < m) y ningún nodo de la SCC tiene aristas que apunten hacia nodos fuera de dicha componente.

2. Guía Metodológica para la Sustentación Oral y Práctica

2.1 Protocolo de Presentación del Software en Vivo

1. **Paso 1 - Demostración HTML (Módulo 2):** Explicar al docente cómo el código parsea código HTML dinámico mediante ExploradorHTML y convierte enlaces en una lista de adyacencia sin recibir datos precalculados.
2. **Paso 2 - Ejecución de Topologías en Consola:** Ejecutar el script y señalar cómo se reportan automáticamente los *Dead Ends*, enlaces duplicados y *Spider Traps* en cada escenario. Mostrar los autovalores |lambda_1|=1.0000 y |lambda_2|=0.8500.
3. **Paso 3 - Validación Métodos (Potencia vs. Monte Carlo):** Destacar que el error en norma L1 entre ambos métodos para 150,000 pasos es inferior a 0.003, demostrando la equivalencia entre la solución determinista y la estocástica.
4. **Paso 4 - Interpretación del Dashboard Gráfico:**
    - **Gráfica 1 (Convergencia):** Mostrar cómo las probabilidades se estabilizan entre 15 y 25 iteraciones.
    - **Gráfica 2 (Comparativa):** Confirmar la coincidencia exacta de barras entre Potencia y Monte Carlo.
    - **Gráfica 3 (Sensibilidad):** Explicar que al aumentar d -> 0.99, la cantidad de iteraciones necesarias aumenta exponencialmente.

- **Gráfica 4 (Ranking SEO - Topología 3):** Demostrar que la página T obtiene el PageRank más alto debido al acumulador M.

2.2 Desarrollo Analítico en Tablero (Topología 1 - 3 Nodos)

Durante la sustentación individual, el docente solicitará resolver analíticamente en tablero la Topología 1 (P1 -> {P2, P3}, P2 -> {P3}, P3 -> {P1}).

Paso A: Matriz de Conteo W y Matriz de Transición Condicional P

Grado de salida: c_1 = 2, c_2 = 1, c_3 = 1.

$$P = \begin{bmatrix} 0 & 1/2 & 1/2 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

Paso B: Construcción de la Matriz de Google M (d = 0.85, 1-d = 0.15, (1-d)/3 = 0.05)

Fórmula: M = 0.85 * P + 0.05 * E

$$M = 0.85 * \begin{bmatrix} 0 & 0.5 & 0.5 \\ 0 & 0 & 1.0 \\ 1.0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0.05 & 0.05 & 0.05 \\ 0.05 & 0.05 & 0.05 \\ 0.05 & 0.05 & 0.05 \end{bmatrix}$$

$$M = \begin{bmatrix} 0.050 & 0.475 & 0.475 \\ 0.050 & 0.050 & 0.900 \\ 0.900 & 0.050 & 0.050 \end{bmatrix}$$

Paso C: Primera Iteración del Método de la Potencia

Vector inicial: pi_0 = [1/3, 1/3, 1/3] = [0.3333, 0.3333, 0.3333].

pi_1 = pi_0 * M = [(0.05+0.05+0.90)/3, (0.475+0.05+0.05)/3, (0.475+0.90+0.05)/3] = [0.3333, 0.1917, 0.4750].

Esto demuestra que el nodo P3 incrementa rápidamente su PageRank por recibir enlaces directos de P1 y P2.

3. Cuestionario de Sustentación Argumentado

A continuación se presentan las respuestas formales, rigurosas y conceptuales para el cuestionario de evaluación en tablero estipulado en la Sección 7 de la guía de laboratorio.

Pregunta 1: ¿De qué manera la adición del término (1-d)(1/m)E garantiza que la matriz M sea irreducible y aperiódica según el Teorema de Perron-Frobenius?

Respuesta Sustentada:

En un grafo web real, la matriz de transición original P puede ser reducible (presencia de

subgrafos desconectados o Spider Traps) o periódica (bucles cerrados de longitud fija). La adición del término de teletransporte (1-d)(1/m)E transforma la matriz de transición P en la Matriz de Google M = d P + (1-d)(1/m)E.

- **Garantía de Irreducibilidad:** Como todos los elementos de E son 1, y (1-d) > 0, la matriz M es estrictamente positiva (M_ij > 0 para todo i, j). Esto implica que existe una probabilidad estrictamente mayor que cero de transicionar directamente entre cualquier par de páginas en un solo paso, eliminando cualquier componente aislada o trampas absortivas.
- **Garantía de Aperiodicidad:** Dado que los elementos diagonales M_ii >= (1-d)/m > 0, cada estado posee un autobucle implícito de probabilidad positiva, lo que rompe cualquier ciclo periódico y garantiza que el máximo común divisor de las longitudes de los caminos de retorno a cualquier estado sea 1.
- **Conclusión según Perron-Frobenius:** Al ser M una matriz estocástica irreducible y aperiódica (estrictamente positiva), el Teorema de Perron-Frobenius asegura que M posee un autovalor dominante único lambda_1 = 1, y que su autovector izquierdo correspondiente pi es estrictamente positivo (pi_j > 0), único y normalizado (sum pi_j = 1), garantizando la existencia y unicidad del vector estacionario de PageRank.

Pregunta 2: Si una página incluye 3 enlaces duplicados a una misma URL, ¿cómo altera esto los pesos en la fila i de la matriz P y qué impacto genera en el PageRank resultante?

Respuesta Sustentada:
En la inspección dinámica de HTML, el peso w_ij representa el número total de hipervínculos dirigidos desde la página i hacia la página j.

- **Alteración en la Fila i de la Matriz P:** Si la página i contiene 3 enlaces duplicados hacia la página objetivo j, el peso en el numerador se incrementa a w_ij = 3. Al mismo tiempo, el grado de salida total c_i = sum_k w_ik incluye estos duplicados en el denominador. Por lo tanto, la probabilidad condicional de transición se convierte en p_ij = 3 / c_i. Esto triplica la proporción de la masa de probabilidad que la página i otorga a la página j en comparación con un único enlace.
- **Impacto en el PageRank Resultante:** Al aumentarse la probabilidad de transición p_ij, la entrada correspondiente en la matriz de Google M_ij = d * p_ij + (1-d)/m también se incrementa significativamente. Como la ecuación estacionaria asigna pi_j = sum_k pi_k * M_kj, un mayor valor en M_ij transfiere una mayor fracción del PageRank pi_i hacia la página objetivo j, inflando su PageRank a expensas de los otros destinos salientes de la página i. Esta es la base matemática de las técnicas de manipulación SEO agresivas.

Pregunta 3: En la Topología 3 (Ataque SEO), ¿por qué la página Megáfono M conserva un PageRank elevado a pesar de no ser la

página objetivo principal?

Respuesta Sustentada:
En la Topología 3, la granja de páginas falsas (F1 ... F8) enlaza masivamente a la página objetivo T (con F8 enviando 3 enlaces duplicados). Sin embargo, la página objetivo T redirige su único enlace saliente hacia la página Megáfono M, y M a su vez distribuye enlaces de retorno hacia todas las páginas falsas F1 ... F8.

- **Mecanismo de Concentración y Transferencia del PageRank:** La página objetivo T acumula una enorme cantidad de flujo entrante proveniente de las 8 páginas falsas (sum_k pi_Fk * M_Fk,T). Sin embargo, como T solo posee un único enlace saliente (c_T = 1, dirigido exclusivamente hacia M), la probabilidad de transición condicional es p_TM = 1.0.
- **Efecto Embudo en la Página M:** Por la ecuación estacionaria: pi_M = pi_T * M_TM + sum_{k != T} pi_k * M_kM. Como M_TM = d(1.0) + (1-d)/10 = 0.85 + 0.015 = 0.865, la página Megáfono M recibe directamente el 86.5% de todo el PageRank acumulado por la página objetivo T.
- **Conclusión Estructural:** Aunque M no es el objetivo de la granja de enlaces, actúa como un "embudo" o drenaje obligatorio de todo el flujo acumulado en T. Al devolver enlaces a las páginas F_i, M realimenta el ciclo cerrado del ataque SEO, conservando un PageRank tan alto como el de la propia página T.

Pregunta 4: ¿Cómo influye el valor del segundo autovalor máximo |lambda_2| de la matriz M en la tasa de convergencia geométrica del Método de la Potencia?

Respuesta Sustentada:
El Método de la Potencia calcula el vector estacionario mediante la iteración pi^{(n)} = pi^{(0)} M^n. Expresando el vector inicial en la base de autovectores de M correspondientes a sus autovalores |lambda_1| > |lambda_2| >= |lambda_3| >= ...:
||pi^{(n)} - pi^{(\infty)}|| se comporta como O(|lambda_2|^n).

- **Relación entre |lambda_2| y el Factor de Amortiguación d:** Un teorema fundamental en la teoría de PageRank establece que los autovalores de la Matriz de Google M = d P + (1-d)(1/m)E satisfacen lambda_1 = 1 y lambda_k(M) = d * lambda_k(P) para todo k >= 2. Por lo tanto, el segundo autovalor máximo cumple: |lambda_2(M)| <= d. En la mayoría de las redes web, |lambda_2(M)| = d = 0.85.
- **Tasa de Convergencia Geométrica:**
    - El término de error decrece geométricamente a una razón de d^n = (0.85)^n por iteración.
    - Para alcanzar una tolerancia de precisión epsilon = 10^-8, el número de iteraciones k requeridas satisface (0.85)^k <= 10^-8, lo que requiere aproximadamente k = 114

iteraciones como límite superior teórico.

- **Efecto de variar d:** Si d -> 1.0 (por ejemplo, d = 0.99), entonces |lambda_2| -> 1.0, haciendo que el factor (0.99)^n decrezca extremadamente despacio, exigiendo cientos o miles de iteraciones para converger. Si d -> 0.10, |lambda_2| = 0.10 y la convergencia ocurre en menos de 5 iteraciones.